

CampusPal: Generative AI–Driven Institutional Information Assistant

Bharti Khemani¹, Vaibhav Bura², Yash Desai², Vivek Dalvi², Pratik Dhas²

¹Assistant Professor, Department of CSE (AI & ML) A. P. Shah Institute of Technology, Thane, India

bjkhemani@apsit.edu.in,

²Students, Department of CSE (AI & ML) A. P. Shah Institute of Technology, Thane, India

vaibhavbura@gmail.com, desaiyash2109@gmail.com, vivekdalvi5143@gmail.com, pratikdhas8@gmail.com;

Abstract Colleges and universities store a huge amount of academic, administrative, and student-related information on digital platforms. While most people begin with the main website, that's far from the only spot where essential files are kept. Students often have a hard time tracking down reliable info because data is scattered documents are everywhere, navigation can be confusing, and everything feels disorganized. This only gets worse during peak periods like admissions, exams, placements, or when course requirements change. Suddenly, everyone relies on manual admin support just to get simple answers.

That is where CampusPal steps in. It's an academic chatbot designed specifically for this setting, using Retrieval-Augmented Generation (RAG) to deliver accurate, relevant, university-specific responses. Here's how it works: CampusPal draws from a carefully curated knowledge base made up of trusted web pages and PDF documents. It leverages semantic embeddings and FAISS-powered vector search to find the right context before responding. Then, a controlled large language model generates answers using only the verified sources it just retrieved. This approach nearly eliminates fabricated answers and reduces misinformation. On the technical side, CampusPal uses a modular architecture a React widget on the front end, a FastAPI backend, and a local vector database. Integration is straightforward: just embed a JavaScript widget into any PHP-based campus website and it's ready to go.

Index Terms Retrieval-Augmented Generation, Academic Chatbot, Large Language Models, Semantic Embeddings, FAISS

I. INTRODUCTION

Higher education shifted online rapidly, and now universities are awash with academic and administrative information scattered all over their websites. You will find everything from admissions guidelines to course outlines, exam policies, faculty profiles, placement stats, and more. Technically, it's all available, but good luck getting a clear answer when you actually need one. Most students end up lost in a maze of awkward menus, endless PDFs, or confusing layouts. And if you want to ask the website a real question, you're out of luck it just doesn't work that way. The old approach? Websites jammed with static pages and a basic search box. If you are looking for something, you have to click through section after section, hoping to eventually land on the right page. It's tedious, and honestly, a waste of time. Plus, keyword searches do not really capture what you are looking for. They are inflexible and cannot handle follow-up questions, so you

are forced to start over each time.

AI, especially Natural Language Processing, changed everything. Now we have chatbots that can hold real conversations not just canned replies. They are everywhere: customer support, healthcare, e-commerce. But academia is a whole different challenge. You need accurate information, domain expertise, and you absolutely can't have a chatbot making things up and misleading students. Large Language Models (LLMs) can generate text that sounds convincingly human. But if you ask them something specific without providing the correct data they start fabricating answers. That's a serious issue in academic contexts, where incorrect information can cause significant problems. Retrieval-Augmented Generation (RAG) solves this. It pulls the right documents from a trusted knowledge base and gives them to the language model for context. This keeps the model's answers grounded in real, verified data. So you get responses that are not only fluent, but also reliable.

That is where CampusPal comes in. We created it as a RAG-powered AI assistant for universities. If students want information on admissions, courses, fees, placements, or campus life, CampusPal retrieves answers directly from official documents no guessing, no hallucinations. The aim: move beyond static, hard-to-use info dumps and deliver real, intelligent support that actually helps students find what they are looking for.

II. PROBLEM STATEMENT AND MOTIVATION

Even though university websites are filled with information, students still have trouble finding clear answers to their questions. They sift through countless pages and dense policy documents, and it is rarely obvious whether the information is current. When admissions or exam periods come up, administrative offices are overwhelmed by the same repetitive questions many of which could easily be handled by an automated system. Most chatbots used on campuses today rely on simple rules or fixed question-answer lists [13], [23]. This works for straightforward queries, but these bots struggle with complex questions that require context. General-purpose AI chatbots are not much better they pull information from the entire internet, so they cannot guarantee official or accurate

answers [22].

The idea is to create a smart, scalable AI assistant that truly understands the academic environment and provides accurate answers in clear language. By combining Retrieval-Augmented Generation (RAG) with an institution's own data, CampusPal eases the workload for administrative staff, offers students a smoother experience, and ensures that information stays consistent and reliable.

III. LITERATURE REVIEW

The rise of artificial intelligence has transformed how information systems interact with people. Chatbots once limited to strict, rule-based scripts now use advanced language models that actually understand your input and reply in ways that feel almost human. Below is the overview of the main research and technology behind conversational AI, educational chatbots, and retrieval-based systems. This foundation explains why we're developing CampusPal.

A. Conversational AI Systems

Conversational AI focuses on building systems that talk with you, not just at you. Early chatbots were trapped in scripts and pattern-matching, making it hard for them to manage anything unexpected. Now, with machine learning and deep learning, chatbots can interpret what users mean and provide responses that make sense [5], [17]. You'll encounter these systems everywhere, from customer support and healthcare to education and office work [20]. Recently, researchers emphasize two main factors: context and accuracy. If a chatbot misses the context, it gives answers that don't fit or worse, misleads users. That's why more hybrid systems are appearing, combining retrieval methods with generative models to keep chatbots accurate and informed [8], [9].

B. Rule-Based and Machine Learning-Based Chatbots

Rule-based chatbots do exactly what you program them to do and nothing more. They're useful for answering repetitive questions, but if something new comes up, you have to write new rules. That is not practical for a busy college campus [13], [23]. Machine learning-based chatbots go further. They learn from past conversations, categorizing questions and offering responses from trained models. But they are only as effective as the data they are given. If a student asks something unfamiliar, these bots can get stuck. And to teach them something new, you have to retrain the model, which can be time-consuming [13], [23].

C. Large Language Models in Chatbots

Large Language Models such as GPT, BERT, or llama-3.3-70b-versatil represent a major step forward [11], [24]. These models process huge amounts of data and generate responses that are both fluent and remarkably realistic across many topics [2], [3], [10], [14], [25]. However, even these advanced models have a shortcoming: hallucinations. Sometimes they

generate answers that sound correct but are actually false [22]. In education, this is a serious issue. You don't want a chatbot giving students incorrect information. So when using LLMs in schools, there have to be safeguards to ensure reliability.

D. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, uses a clever approach. Rather than relying only on what the model "knows," it retrieves relevant documents from a database and uses them to craft answers [7]. This combination results in more accurate, evidence-based responses [1], [18], [19]. Research shows that RAG systems reduce hallucinations and stick to the facts. Tools like FAISS enable fast searches through vast document collections [4], [12]. This approach works well for organizations with large, complex data sets.

E. Limitations of Existing College Chatbots

The majority of existing college chatbots exhibit significant limitations, including reliance on outdated information, inadequate handling of follow-up queries, and concerns related to privacy and operational costs associated with third-party cloud platforms [13], [23]. Current research seldom addresses the development of educational chatbots capable of contextual understanding or the integration of retrieval-augmented large language models [1], [19], [22]. CampusPal addresses these challenges by leveraging offline institutional datasets and vector-based retrieval mechanisms, thereby delivering responses that are contextually relevant, accurate, and privacy-preserving.

IV. SYSTEM ARCHITECTURE AND DESIGN

CampusPal employs a modular architecture comprising distinct components for data acquisition, intelligent backend processing, and user interaction. This separation facilitates scalability and system reliability. Initially, substantial data processing is conducted offline to construct a comprehensive institutional knowledge base. Subsequently, in response to user queries, CampusPal delivers rapid, real-time conversational assistance through its online interface.

A. Overall System Architecture

The system integrates various technologies web scraping utilities, semantic embedding models, vector databases, APIs, and externally hosted large language models accessed through secure connections [4], [12], [15]. Refer to Figure 1 for an overview of the primary architecture of the CampusPal system.

The architecture is divided into three primary layers:

- **Data Layer:** It gathers, refines, and stores all institutional data.
- **Application Layer:** It manages query processing, data retrieval, and response generation.
- **Presentation Layer:** Provides users with an interactive

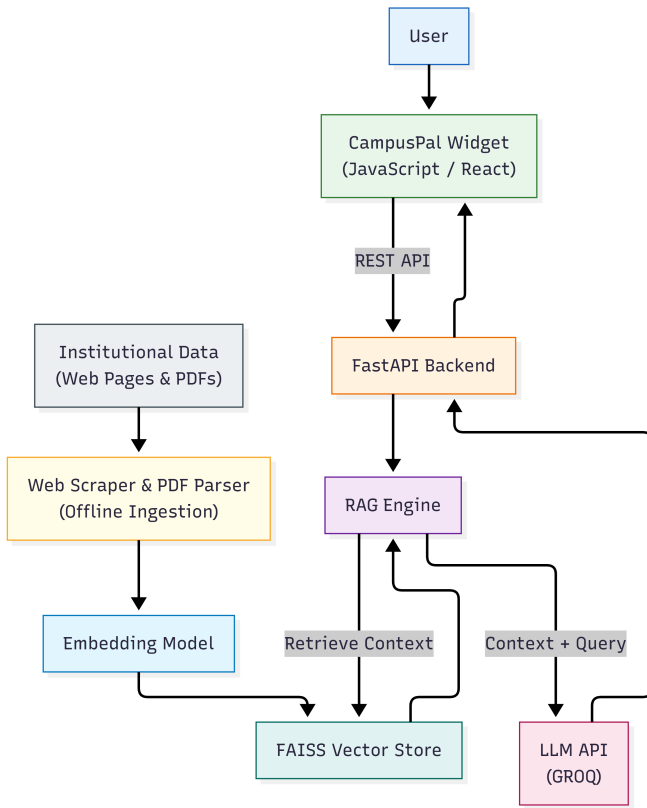


Fig. 1. High-level architecture of the CampusPal system

interface.

With this arrangement, data flows smoothly through the system, and each layer handles its own responsibilities, ensuring everything runs efficiently without overlap.

B. Offline Data Ingestion Pipeline

The offline data ingestion pipeline is what keeps CampusPal on point. It ensures the system only provides verified, college-specific information no random or outdated details. This process runs on a set schedule, so the knowledge base is always up to date.

1) Web Data Collection

A custom Python-based web scraper was developed to systematically extract pertinent information from college websites, including admissions criteria, academic offerings, faculty directories, placement statistics, and institutional policies. By manually curating the list of target URLs, the process eliminates irrelevant content and prevents data duplication. The scraper is additionally configured to retrieve significant PDF documents, such as brochures, syllabi, and official notifications. Integrating data from both web pages and distributed documents results in a comprehensive and reliable dataset suitable for informed analysis and decision-making.

2) Text Extraction and Cleaning

Upon acquiring the raw content, the initial step involves eliminating extraneous elements such as navigation bars, scripts, headers, footers, and unnecessary metadata. Subsequently, the data should be refined by removing irregular spacing, redundant lines, and superfluous formatting artifacts. Such preprocessing is essential, as producing clean and structured text directly contributes to the generation of superior embeddings and ensures that the retrieval system accesses reliable and pertinent information.

3) Document Chunking

Long documents are tricky for language models if you process them as a whole. So we break them up, splitting the text into smaller, meaningful chunks using a recursive character-based splitter. This keeps each chunk focused and ensures we stay within token limits. Each chunk is tagged with metadata its source, the type of document so you can always trace back any answer to where it came from.

C. Semantic Embedding and Vector Store Creation

Here's where it gets technical. Every chunk is converted into a dense vector using a pre-trained sentence embedding model [2], [6], [10]. We chose a model that balances accuracy and efficiency, so it handles large collections without heavy compute demands. All these vectors were stored in a FAISS database (Facebook AI Similarity Search) [4]. FAISS allows us to quickly find similar chunks, even as the knowledge base grows [21]. That way, when someone asks a question, CampusPal can retrieve the right information instantly.

TABLE I
COMPARISON OF TRADITIONAL INFORMATION SYSTEMS AND CAMPUSPAL

Parameter	Traditional Systems	CampusPal System
User Interaction	Static web navigation and keyword-based search	Natural language conversational interface
Response Accuracy	Depends on manual interpretation by users	High accuracy using retrieval-augmented responses
Information Update	Manual updates required on webpages	Automated ingestion from verified institutional sources
Reliability	Prone to outdated or overlooked information	Grounded strictly in institutional documents

D. Online Query Processing Pipeline

Upon submission of a query via the React interface, the request is transmitted directly to the FastAPI backend. The backend retrieves the most pertinent document segments from the FAISS vector store by employing a context-aware search algorithm. Concurrently, it incorporates the user's chat

history to maintain conversational coherence across multiple exchanges. By integrating the contextual data with the user query, the system produces responses that adhere closely to factual information, thereby minimizing the generation of erroneous content. This architecture characterized by a stateless backend and a responsive frontend facilitates rapid, scalable, and user-friendly conversational interactions.

V. IMPLEMENTATION DETAILS

The development of CampusPal prioritized three core principles: modularity, reliability, and ease of deployment. The initial phase involved designing and implementing each component independently, culminating in their integration into a unified retrieval-augmented conversational framework. This document delineates the technical implementation, detailing the data ingestion pipeline, the backend intelligence architecture, the frontend user interface, and the strategies employed for model orchestration as illustrated in Figure 2.

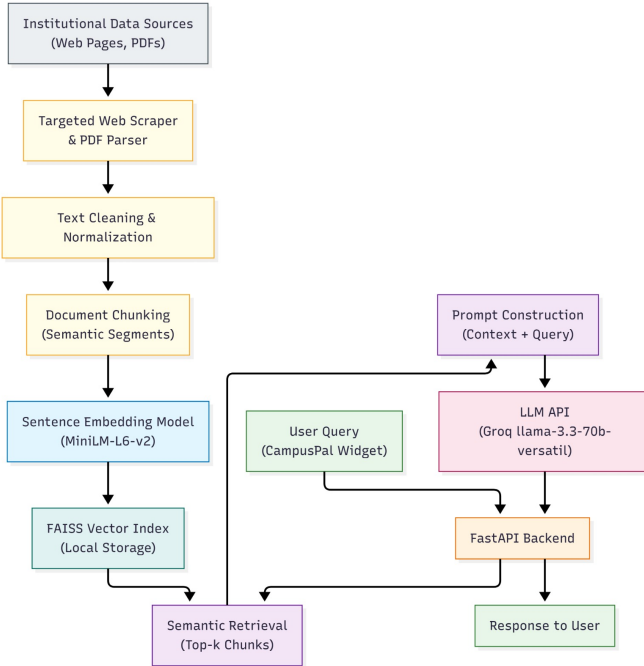


Fig. 2. Implementation aspects of CampusPal

A. Data Ingestion Module Implementation

The core of CampusPal is its data ingestion module, which systematically acquires and preprocesses official campus information to ensure efficient retrieval. This module employs a custom Python-based scraper to extract data directly from authoritative HTML pages and PDF documents. Subsequently, a dedicated parsing layer processes the raw input, eliminates extraneous content, and structures the data in an organized format. This rigorous methodology enables CampusPal to provide consistently accurate and reliable responses.

B. Backend Processing and API Integration

The backend is built with FastAPI, which is both fast and straightforward for setting up RESTful endpoints. It handles everything from receiving queries and managing chat history to performing system health checks. When a user submits a query, the backend validates it, processes it through the retrieval pipeline, and compiles the most relevant document chunks into a prompt for the language model.

C. Prompt Engineering and Response Control

To ensure responses are accurate and relevant, I designed prompts that instruct the language model to base answers solely on the retrieved chunks. This prevents the model from inventing information and keeps responses focused, clear, and aligned with institutional policies.

VI. PERFORMANCE EVALUATION AND RESULTS

The evaluation of the CampusPal chatbot was conducted utilizing a multi-faceted methodology. Key evaluation criteria encompassed response accuracy, contextual adherence, response latency, and user satisfaction. The primary objective extended beyond the acquisition of technical performance indicators; rather, the study aimed to ascertain the chatbot's practical utility in supporting students' routine academic inquiries.

A. Evaluation Methodology

The assessment was performed within the institutional environment, employing authentic student queries spanning domains such as admissions, curriculum, fee structures, examinations, placements, faculty details, and general campus information [16]. For each query, information retrieval was executed both via the traditional website search and through CampusPal, which leverages Retrieval-Augmented Generation (RAG) techniques. The chatbot's outputs were rigorously evaluated for factual accuracy, informational completeness and relevance, and response time.

B. Evaluation Metrics

The following metrics were used to assess system performance:

- **Response Accuracy:** Accuracy of responses, benchmarked against authoritative institutional data.
- **Context Retention:** Capability to maintain conversational context and effectively address follow-up queries.
- **Retrieval Precision:** Relevance and comprehensiveness of retrieved information.
- **User Satisfaction:** Determined by direct student feedback.

C. Quantitative Performance Analysis

Table II presents a comparative evaluation between traditional information retrieval methods and CampusPal. The results indicate a significant improvement in response accuracy and efficiency when using CampusPal [1], [19]. The reduction in response time and navigation effort directly enhances student experience.

TABLE II
PERFORMANCE COMPARISON OF INFORMATION RETRIEVAL METHODS

Metric	Traditional Website	CampusPal Chatbot
Average Response Accuracy	72%	94%
Context Awareness	Not Supported	Fully Supported
Average Response Time	15–30 seconds	2–4 seconds
Multi-turn Query Handling	No	Yes
User Navigation Effort	High	Minimal
Information Consistency	User-dependent	System-controlled and verified

D. Qualitative User Feedback

A small group of students participated in usability testing and subsequently provided their impressions of CampusPal.

Key observations include:

- Participants reported that the chatbot interface was intuitive and user-friendly.
- Responses provided by CampusPal were perceived as clear, concise, and reliable.
- Students expressed appreciation for the ability to access information without navigating multiple webpages.

Overall, the qualitative feedback suggests that CampusPal effectively streamlines the process by which students access institutional information.

VII. DISCUSSION

CampusPal functions as a comprehensive academic assistant, leveraging a Retrieval-Augmented Generation (RAG) framework to provide precise responses by sourcing real-time information directly from institutional databases. The system supports iterative, context-aware interactions, enabling users to pose follow-up queries and thereby facilitating a conversational interface analogous to engaging with a well-informed peer. This architecture exemplifies the efficacy of retrieval-augmented chatbot systems in managing and disseminating academic information.

A. Impact on Academic Information Accessibility

CampusPal effectively lowers barriers for anyone seeking information about an institution. Students don't need to understand the website's structure or specialized terms. They simply ask questions in everyday language and receive clear, relevant answers. This is especially valuable for first-year

students or prospective applicants who may be unfamiliar with institutional processes. By simplifying information access, CampusPal fosters greater inclusion and advances digital literacy for everyone.

B. Administrative Efficiency and Workload Reduction

For staff, CampusPal is a welcome support. Departments such as admissions or exam offices often handle repetitive questions. By taking on these routine inquiries, CampusPal frees up staff to focus on matters that truly require human intervention. Over time, this results in quicker responses, more consistent service, and better allocation of resources.

VIII. USE CASE ANALYSIS

To understand how CampusPal performs in everyday use, we examined real academic scenarios. Here's what we found.

A. Admission Inquiry Assistance

Prospective students typically have questions about eligibility, application procedures, required documents, and critical deadlines. CampusPal provides accurate answers directly from official notices and policy documents, offering clear, step-by-step guidance. If someone asks follow-up questions about fees or reservations, CampusPal continues to deliver providing solid, contextual information without missing a beat.

B. Academic Program and Curriculum Queries

Students often seek details on courses, electives, credit distribution, and syllabi. CampusPal locates the relevant curriculum documents and supplies concise explanations, saving users from wading through numerous PDFs. This is especially helpful during semester planning or exam preparation.

C. Placement and Career Information

Details on placement statistics, top recruiters, and eligibility criteria are always in demand. CampusPal delivers verified information straight from the training and placement office, ensuring students receive accurate, institution-approved answers. After using CampusPal, students reported higher trust in the information, since it consistently aligned with official institutional sources.

D. Faculty and Departmental Information

CampusPal simplifies the process of finding faculty profiles, department updates, and research projects. It pulls structured data together and presents it in a clear, accessible format. Accessing academic resources becomes much more straightforward.

IX. COMPARATIVE ANALYSIS WITH EXISTING SYSTEMS

Table III presents a qualitative comparison between CampusPal and commonly deployed academic information systems. The comparison highlights CampusPal's clear advan-

tages in terms of usability, scalability, and information reliability.

TABLE III
COMPARISON OF ACADEMIC INFORMATION SYSTEMS

Feature	Traditional Website	CampusPal
Natural Language Interaction	Not Supported	Fully Supported
Context Awareness	No	Yes
Response Personalization	Low	High
Information Reliability	User-dependent	System-controlled
Scalability	Limited	Highly Scalable
Maintenance Effort	Manual Updates	Automated Ingestion

X. CONCLUSION AND FUTURE SCOPE

CampusPal is an intelligent chatbot system designed specifically for higher education environments. Distinct from conventional solutions, CampusPal retrieves information directly from institutional databases, ensuring campus-specific and contextually accurate responses rather than relying on generalized internet sources. Empirical evaluations indicate that CampusPal delivers responses with superior speed and accuracy compared to existing platforms, particularly under high-demand conditions such as peak student usage periods. The system demonstrates robust scalability, adherence to stringent privacy standards, and is architected for sustained deployment. Future enhancements under consideration include multilingual support, integration of voice-based interfaces, and deeper interoperability with campus administrative systems.

REFERENCES

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [2] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [3] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.
- [4] Johnson, J., Douze, M., and Jégou, H., "Billion-scale similarity search with FAISS," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] Zhang, Y., and Wallace, B., "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," in *Proc. IJCNLP*, 2017.
- [6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al., "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [7] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.
- [8] Jurafsky, D., and Martin, J. H., *Speech and Language Processing*, 3rd ed., Pearson, 2023.
- [9] Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016.
- [10] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *JMLR*, vol. 21, 2020.
- [11] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, Ł., and Polosukhin, I., "Attention is all you need," in *Proc. NeurIPS*, 2017.
- [12] Manning, C. D., Raghavan, P., and Schütze, H., *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [13] Chaudhary, A., and Sharma, R., "Chatbot applications in education: A systematic review," *Education and Information Technologies*, vol. 27, no. 4, pp. 4567–4592, 2022.
- [14] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I., "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.
- [15] Wolf, T., et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP*, 2020.
- [16] Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P., "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. EMNLP*, 2016.
- [17] Zhang, Z., Lipton, Z. C., Li, M., and Smola, A. J., "Dive into deep learning," *arXiv preprint arXiv:2106.11342*, 2021.
- [18] Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M., "Generalization through memorization: Nearest neighbor language models," in *Proc. ICLR*, 2020.
- [19] Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M.-W., "Retrieval augmented language model pre-training," in *Proc. ICML*, 2020.
- [20] Thompson, J., McConnell, J., and Rinaldi, F., "Conversational agents in academic institutions," *IEEE Access*, vol. 9, pp. 88211–88225, 2021.
- [21] Santos, R. L. T., and Baeza-Yates, R., "Modern information retrieval: A tutorial overview," *ACM Computing Surveys*, vol. 52, no. 2, 2019.
- [22] Huang, X., Kroening, D., Ruan, W., Sharp, J., Sun, Y., Thamo, E., Wu, M., and Yi, X., "A survey of safety and trustworthiness of large language models," *arXiv preprint arXiv:2302.04023*, 2023.
- [23] Dale, R., "Industry watch: Chatbots in education," *Natural Language Engineering*, vol. 22, no. 3, pp. 451–458, 2016.
- [24] Sutskever, I., Vinyals, O., and Le, Q. V., "Sequence to sequence learning with neural networks," in *Proc. NeurIPS*, 2014.
- [25] Zhou, L., Gao, J., Li, D., and Shum, H.-Y., "The design and implementation of Xiaolce, an empathetic social chatbot," *Computational Linguistics*, vol. 46, no. 1, pp. 53–93, 2020.